LAPORAN UJIAN TENGAH SEMESTER
DEEP LEARNING



Dosen pengampu:
Dr. Dra. Dwina Kuswardani, M.Kom

Disusun Oleh:
Tarina Angeline Aurelia (202432074)

FAKULTAS TELEMATIKA ENERGI
SISTEM INFORMASI
INSTITUT TEKNOLOGI PLN
TAHUN 2026/2027

BAB 1
LATAR BELAKANG MASALAH & EKSPLORASI DATASET

1.1 Latar Belakang Masalah

Transportasi darat adalah tulang punggung mobilitas perkotaan. Volume kendaraan pada ruas jalan arteri tidak konstan, tetapi berubah dari jam ke jam mengikuti pola aktivitas manusia: jam sibuk pagi dan sore pada hari kerja, pola yang berbeda pada akhir pekan, hari libur nasional, serta kondisi cuaca. Interstate 94 (I-94) adalah jalan bebas hambatan yang menghubungkan Minneapolis dan St Paul di negara bagian Minnesota, Amerika Serikat, dan menjadi salah satu koridor yang padat di kawasan tersebut.

Peramalan volume lalu lintas jangka pendek memberikan manfaat praktis bagi pengelola jalan, misalnya untuk perencanaan manajemen lalu lintas, pengaturan sinyal, penjadwalan pemeliharaan, dan antisipasi kemacetan. Tantangannya, pola volume lalu lintas bersifat nonlinier, berulang pada siklus harian (24 jam) dan mingguan (168 jam), serta dipengaruhi faktor eksternal. Karakter data yang bergantung pada urutan waktu ini membuat pendekatan Deep Learning berbasis sekuens menjadi kandidat yang sesuai.

Kasus **forecasting time-series** dipilih karena digit terakhir NIM penulis (202432074) adalah **genap**, sehingga sesuai pembagian topik pada instruksi ujian. Algoritma utama yang dipilih adalah **LSTM (Long Short-Term Memory)**. Gerbang forget, input, dan output beserta cell state membuat LSTM mampu menyimpan informasi jangka panjang secara selektif dan lebih tahan terhadap masalah vanishing gradient dibandingkan RNN standar, sehingga cocok untuk pola harian yang berulang setiap 24 langkah waktu.

1.2 Tujuan

Tujuan dari proyek ini adalah:

- Membangun model LSTM yang memprediksi **traffic_volume 1 jam ke depan** berdasarkan **24 jam terakhir**.
- Mengevaluasi model menggunakan RMSE, MAE, dan R^2 pada data uji, serta membandingkannya dengan dua baseline sederhana (persistence dan seasonal naive).
- Membuktikan pemahaman alur komputasi internal LSTM melalui perhitungan manual 1 nilai input dan 1 nilai output pada 1 langkah waktu.

- Memvisualisasikan riwayat loss pelatihan serta perbandingan data aktual dan prediksi pada data uji, lalu menganalisisnya.

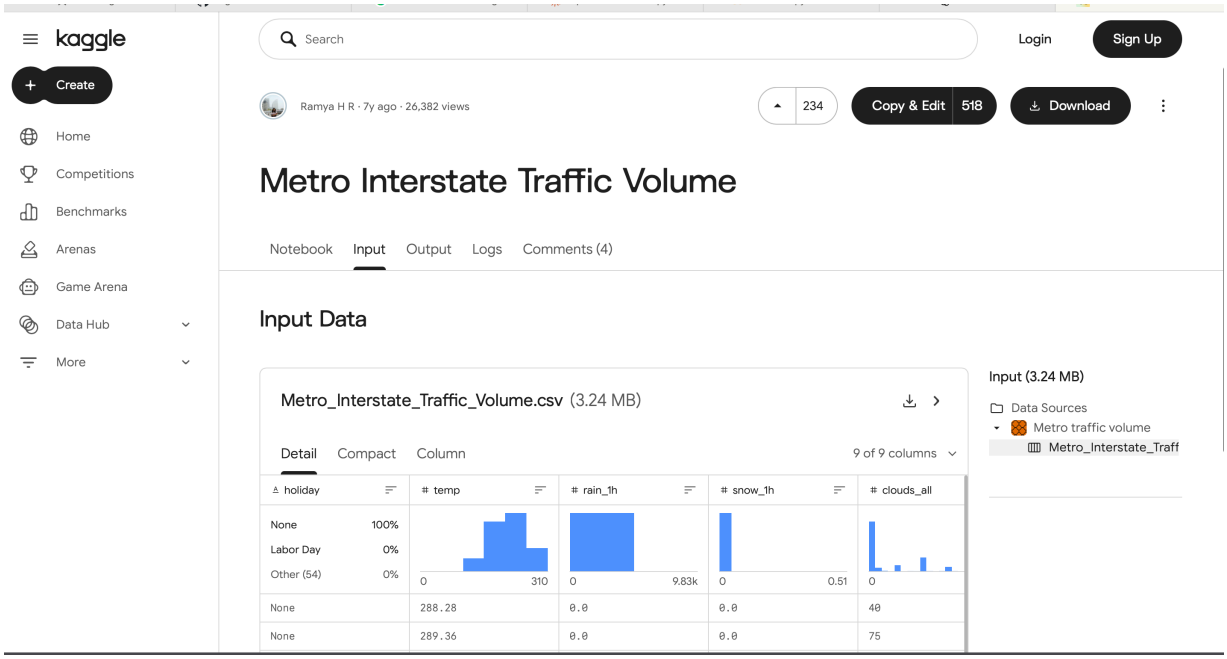
1.3 Eksplorasi Dataset

1.3.1 Sumber dan Bukti Klaim Dataset

Dataset yang digunakan adalah **Metro Interstate Traffic Volume**. Tautan dataset pada Kaggle:

https://www.kaggle.com/code/ramyahr/metro-interstate-traffic-volume/data

Data asli dikumpulkan dari stasiun pencacah lalu lintas otomatis (ATR) nomor 301 milik Minnesota Department of Transportation yang mengukur volume lalu lintas I-94 arah barat antara Minneapolis dan St Paul, digabungkan dengan data cuaca dan hari libur (Hogue, 2019).



Gambar 1. Tangkapan layar bukti validasi/klaim dataset pada situs Kaggle

1.3.2 Ukuran dan Cakupan Data

Dataset berisi **48.204 baris** dan **9 kolom (fitur)**. Rentang waktu pengamatan adalah **2 Oktober 2012 hingga 30 September 2018** dengan frekuensi pencatatan per jam. Perlu dicatat bahwa pencatatan tidak sepenuhnya rapat: terdapat jam yang tidak tercatat serta beberapa timestamp yang muncul lebih dari satu kali. Kedua hal ini ditangani pada tahap pra-pemrosesan (Bab 3).

1.3.3 Penjelasan Atribut dan Label

Atribut pada dataset Metro Interstate Traffic Volume

No	Atribut	Tipe	Deskripsi	Peran
1	holiday	Kategorikal	Nama hari libur nasional/regional AS (termasuk Minnesota State Fair); bernilai "None" jika bukan hari libur.	Tidak dipakai
2	temp	Numerik	Suhu rata-rata dalam satuan Kelvin.	Tidak dipakai
3	rain_1h	Numerik	Curah hujan dalam satu jam (mm).	Tidak dipakai
4	snow_1h	Numerik	Curah salju dalam satu jam (mm).	Tidak dipakai
5	clouds_all	Numerik	Persentase tutupan awan (%).	Tidak dipakai
6	weather_main	Kategorikal	Deskripsi singkat kondisi cuaca (mis. Clouds, Rain, Snow).	Tidak dipakai
7	weather_description	Kategorikal	Deskripsi cuaca yang lebih rinci.	Tidak dipakai
8	date_time	Datetime	Tanggal dan jam pengambilan data (waktu lokal).	Indeks waktu
9	traffic_volume	Numerik	Volume lalu lintas per jam pada I-94 arah barat (kendaraan/jam).	Target (label)

1.3.4 Fitur yang Digunakan

Pada proyek ini dilakukan **forecasting univariat**: hanya kolom **date_time** (sebagai indeks waktu) dan **traffic_volume** (sebagai target) yang digunakan. Dengan demikian, setiap langkah waktu pada model hanya memiliki 1 fitur input (input_size = 1) sehingga fokus penelitian tertuju pada kemampuan LSTM menangkap pola temporal dari riwayat nilai target itu sendiri. Fitur cuaca dan hari libur tidak digunakan dan dapat menjadi pengembangan lanjutan.

BAB 2
PEMAHAMAN ALGORITMA & PERHITUNGAN MANUAL (1 INPUT & 1 OUTPUT)

2.1 Konsep Dasar RNN dan Keterbatasannya

Recurrent Neural Network (RNN) memproses data sekuens langkah demi langkah dengan membawa sebuah hidden state yang merangkum informasi langkah-langkah sebelumnya. Pada setiap langkah waktu t, RNN standar menghitung:

$$h_t = tanh(W_x \cdot x_t + W_h \cdot h_{t-1} + b)$$

Karena informasi lama harus melewati perkalian matriks dan fungsi tanh berulang kali pada setiap langkah, gradien pada proses pelatihan (backpropagation through time) cenderung mengecil menuju nol pada sekuens panjang. Masalah ini dikenal sebagai vanishing gradient (Bengio dkk., 1994) sehingga RNN sulit mengingat ketergantungan jangka panjang.

2.2 Konsep Kerja dan Arsitektur LSTM

LSTM (Hochreiter & Schmidhuber, 1997) mengatasi keterbatasan tersebut dengan menambahkan **cell state** (C_t) sebagai jalur memori jangka panjang yang hanya dimodifikasi secara terkontrol oleh tiga **gerbang (gate)**. Setiap gerbang adalah lapisan sigmoid (σ) yang menghasilkan nilai 0 sampai 1 sebagai "katup": nilai mendekati 0 berarti menutup aliran informasi, nilai mendekati 1 berarti meloloskannya. LSTM memiliki dua keadaan yang dibawa antar langkah waktu, yaitu cell state C_t (memori jangka panjang) dan hidden state h_t (keluaran/memori jangka pendek).

Komponen LSTM sesuai gambar referensi pada instruksi

Komponen	Rumus	Fungsi	Aktivasi
Forget gate	$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$	Memutuskan seberapa besar isi memori lama C_{t-1} dipertahankan.	Sigmoid
Input gate	$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$	Memutuskan seberapa besar informasi baru boleh masuk ke memori.	Sigmoid
Candidate cell state	$\tilde{C}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_C)$	Menyusun kandidat informasi baru yang berpotensi disimpan.	Tanh
Update cell state	$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$	Memperbarui memori: sebagian memori lama + sebagian memori baru.	–

Komponen	Rumus	Fungsi	Aktivasi
Output gate	$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$	Memutuskan seberapa besar isi memori ditampilkan sebagai keluaran.	Sigmoid
Hidden state	$h_t = o_t * \tanh(C_t)$	Keluaran langkah waktu t (juga diteruskan ke langkah t+1).	Tanh

Alur komputasi satu langkah waktu pada LSTM adalah sebagai berikut.

1. **Penggabungan masukan.** Hidden state sebelumnya h_{t-1} digabung (concatenate) dengan input saat ini x_t menjadi vektor $[h_{t-1}, x_t]$ yang dipakai oleh semua gerbang.
2. **Forget gate** menghitung f_t, yaitu faktor pengali untuk memori lama C_{t-1}.
3. **Input gate dan candidate** menghitung i_t dan $\tilde{C}_t$, yaitu seberapa banyak dan apa informasi baru yang akan ditulis.
4. **Pembaruan cell state**: $C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$. Operasi tambah inilah yang membuat gradien dapat mengalir lebih stabil.
5. **Output gate** menghitung o_t, yaitu seberapa banyak isi memori yang dikeluarkan.
6. **Hidden state** $h_t = o_t * \tanh(C_t)$ dihasilkan, lalu (h_t, C_t) diteruskan ke langkah waktu berikutnya.

Pada proyek ini, langkah-langkah tersebut diulang sebanyak **24 kali** (24 jam masukan). Hidden state pada langkah ke-24 merupakan ringkasan seluruh sekuens dan diteruskan ke lapisan dense untuk menghasilkan prediksi volume lalu lintas jam ke-25.

2.3 Perhitungan Manual: 1 Input (x_t) dan 1 Output (h_t) pada 1 Time Step

Untuk membuktikan pemahaman alur komputasi internal LSTM, berikut dilakukan perhitungan manual sederhana dengan **satu nilai input** x_t yang menghasilkan **satu nilai output** h_t pada satu langkah waktu. Agar dapat dihitung dengan tangan, digunakan satu unit hidden (semua besaran berupa skalar). Nilai bobot dan bias berikut adalah **nilai ilustrasi**, sedangkan pada model sebenarnya bobot diperoleh dari proses pelatihan dengan 64 unit hidden. Pembulatan dilakukan pada 4 angka desimal di setiap langkah. Karena $[h_{t-1}, x_t]$ berisi dua elemen, setiap bobot gerbang terdiri dari dua angka: [bobot untuk h_{t-1}, bobot untuk x_t].

Nilai awal dan parameter perhitungan manual

Besaran	Nilai	Keterangan
Input x_t	0,6	Nilai traffic_volume yang telah dinormalisasi Min-Max (rentang 0–1).
Hidden state h_{t-1}	0,5	Keluaran langkah waktu sebelumnya.
Cell state C_{t-1}	0,4	Memori jangka panjang sebelumnya.
W_f, b_f	[0,5 ; 0,8], 0,1	Bobot dan bias forget gate.
W_i, b_i	[0,4 ; 0,9], 0,2	Bobot dan bias input gate.
W_c, b_C	[0,3 ; 0,7], 0,1	Bobot dan bias candidate cell state.
W_o, b_o	[0,6 ; 0,5], 0,05	Bobot dan bias output gate.

Langkah 1 — Forget gate

$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) = \sigma(0{,}5 \times 0{,}5 + 0{,}8 \times 0{,}6 + 0{,}1)$

$f_t = \sigma(0.8300) = 0.6964$

Langkah 2 — Input gate

$i_t = \sigma(0{,}4 \times 0{,}5 + 0{,}9 \times 0{,}6 + 0{,}2) = \sigma(0.9400) = 0.7191$

Langkah 3 — Candidate cell state

$\tilde{C}_t = \tanh(0{,}3 \times 0{,}5 + 0{,}7 \times 0{,}6 + 0{,}1) = \tanh(0.6700) = 0.5850$

Langkah 4 — Pembaruan cell state

$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t = 0.6964 \times 0{,}4 + 0.7191 \times 0.5850$

$C_t = 0.2786 + 0.4207 = 0.6993$

Langkah 5 — Output gate

$o_t = \sigma(0{,}6 \times 0{,}5 + 0{,}5 \times 0{,}6 + 0{,}05) = \sigma(0.6500) = 0.6570$

Langkah 6 — Hidden state (output)

$h_t = o_t * \tanh(C_t) = 0.6570 \times \tanh(0.6993) = 0.6570 \times 0.6039 = 0.3968$

Ringkasan hasil perhitungan manual LSTM

Besaran	Hasil	Makna
f_t	0.6964	Sekitar 70% memori lama dipertahankan.
i_t	0.7191	Sekitar 72% kandidat informasi baru dimasukkan.
$\tilde{C}_t$	0.5850	Kandidat informasi baru.
C_t	0.6993	Memori baru yang diteruskan ke langkah t+1.

Besaran	Hasil	Makna
o_t	0.6570	Sekitar 66% memori ditampilkan.
h_t (output)	0.3968	Keluaran langkah waktu t.

Hasil menunjukkan bahwa dengan input $x_t = 0{,}6$, LSTM menghasilkan output $h_t = 0.3968$. Nilai h_t selalu berada di rentang −1 sampai 1 karena merupakan hasil kali o_t (0–1) dengan $\tanh(C_t)$ (−1 sampai 1). Sebagai pembanding, RNN standar dengan input dan keadaan yang sama ($W_x = 0{,}8$; $W_h = 0{,}5$; $b = 0{,}1$) langsung menghitung $h_t = \tanh(0{,}8 \times 0{,}6 + 0{,}5 \times 0{,}5 + 0{,}1) = \tanh(0.8300) = 0.6805$ tanpa gerbang dan tanpa cell state. Pada LSTM, jumlah informasi yang lewat dikendalikan oleh f_t, i_t, dan o_t.

2.4 Dari Hidden State ke Nilai Prediksi

Pada implementasi, hidden state langkah terakhir dipetakan oleh lapisan Linear (dense) menjadi satu nilai prediksi pada skala ternormalisasi: $\hat{y}' = W_{fc} \cdot h_t + b_{fc}$. Sebagai ilustrasi, dengan $W_{fc} = 0{,}9$ dan $b_{fc} = 0{,}05$ maka $\hat{y}' = 0{,}9 \times 0.3968 + 0{,}05 = 0.4071$. Nilai ini dikembalikan ke satuan kendaraan/jam dengan kebalikan normalisasi Min-Max:

$$\hat{y} = \hat{y}' \times (x_{max} - x_{min}) + x_{min}$$

dengan x_{min} dan x_{max} adalah nilai minimum dan maksimum data latih yang disimpan oleh objek MinMaxScaler.

2.5 Keterkaitan dengan Implementasi nn.LSTM pada PyTorch

=Rumus pada instruksi memakai satu matriks bobot untuk vektor gabungan $[h_{t-1}, x_t]$. PyTorch memecahnya menjadi dua matriks, yaitu W_{ih} (untuk input) dan W_{hh} (untuk hidden state), serta dua vektor bias (b_{ih} dan b_{hh}) untuk masing-masing dari empat blok (forget, input, candidate, output). Secara matematis keduanya setara karena $W \cdot [h, x] = W_h \cdot h + W_x \cdot x$ dan $b = b_{ih} + b_{hh}$. Pada model dengan input_size = 1 dan hidden_size = 64, jumlah parameter LSTM adalah:

$$4 \times (64 \times 1 + 64 \times 64 + 2 \times 64) = 4 \times 4.288 = 17.152$$

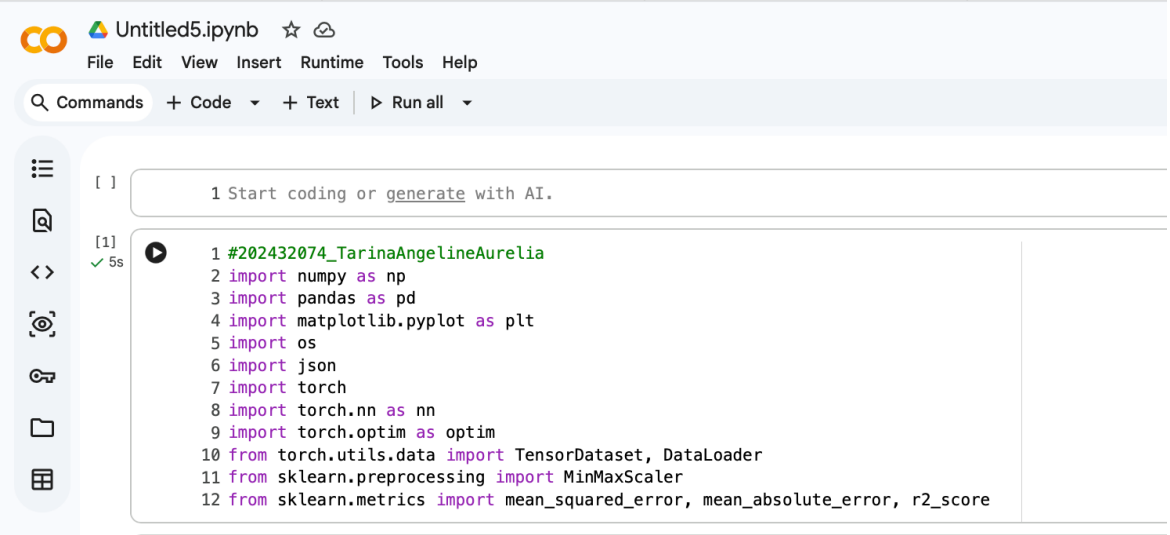
Ditambah lapisan Linear (64 × 1 + 1 = 65 parameter), total parameter model adalah **17.217**. Angka ini dapat dicocokkan dengan keluaran "Total parameter" pada sel pelatihan di Bab 3.

BAB 3
IMPLEMENTASI PRAKTIKUM & PENJELASAN BARIS KODE

Implementasi dilakukan pada Jupyter Notebook menggunakan Python dengan kerangka kerja PyTorch. Berkas notebook mengikuti template yang disediakan dan setiap sel kode diberi watermark #202432074_TarinaAngelineAurelia pada baris paling atas. Notebook dan dataset tersedia pada repositori GitHub.

3.1 Import Library

Sel pertama memuat seluruh pustaka yang dibutuhkan.

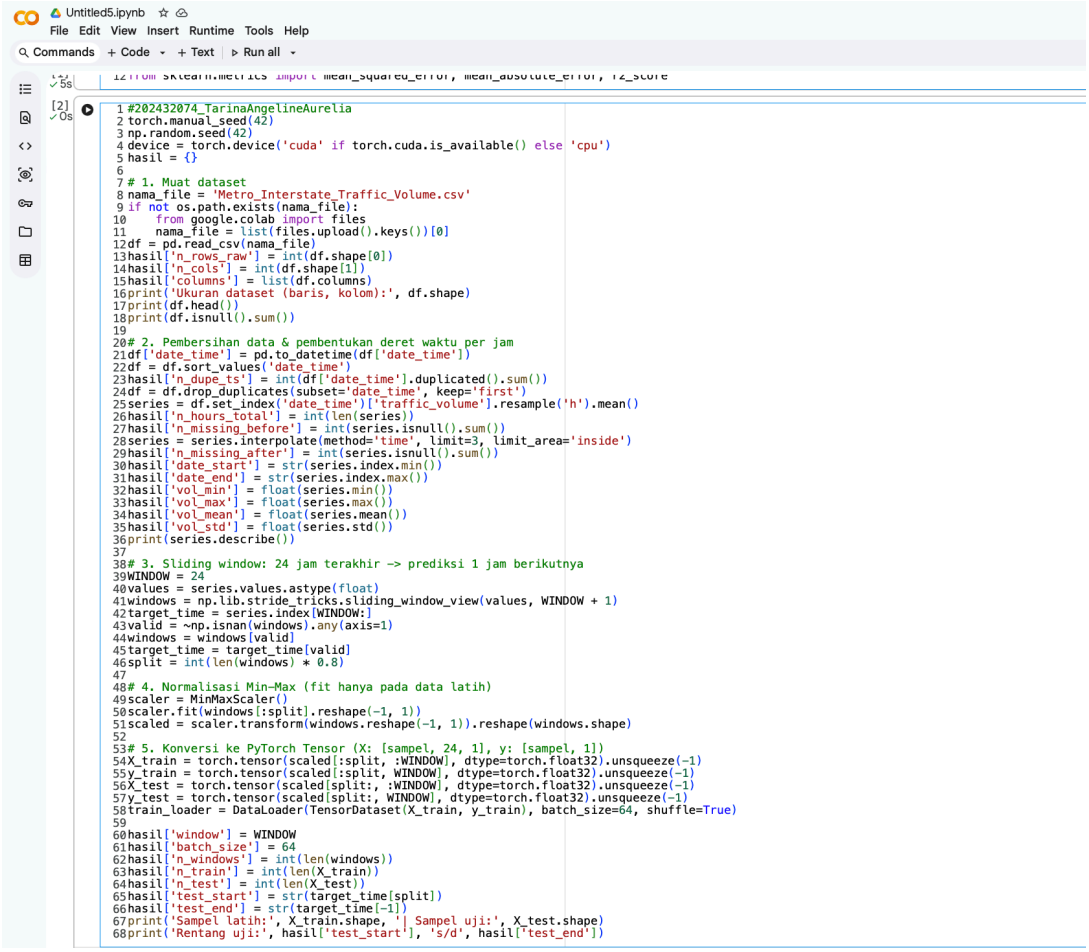


Penjelasan :

No	Baris Kode	Penjelasan
1	import numpy as np	Mengimpor NumPy untuk operasi array numerik; dipakai untuk seed acak, sliding window, dan perhitungan RMSE.
2	import pandas as pd	Mengimpor pandas untuk membaca CSV, mengelola DataFrame, dan mengolah deret waktu (resample, interpolate).
3	import matplotlib.pyplot as plt	Mengimpor modul pyplot dari matplotlib untuk membuat grafik.
4	import os	Mengimpor modul os untuk fungsi sistem berkas; di sini dipakai untuk memeriksa keberadaan file dataset.
5	import json	Mengimpor modul json untuk menyimpan ringkasan hasil ke berkas JSON.

No	Baris Kode	Penjelasan
6	import torch	Mengimpor PyTorch, kerangka kerja tensor dan deep learning yang dipakai.
7	import torch.nn as nn	Mengimpor torch.nn (alias nn) yang berisi layer dan loss function, mis. nn.LSTM, nn.Linear, nn.MSELoss.
8	import torch.optim as optim	Mengimpor torch.optim (alias optim) yang berisi algoritma optimasi, mis. Adam.
9	from torch.utils.data import TensorDataset, DataLoader	Mengimpor TensorDataset (membungkus pasangan X dan y) dan DataLoader (membagi data menjadi mini-batch).
10	from sklearn.preprocessing import MinMaxScaler	Mengimpor MinMaxScaler untuk normalisasi data ke rentang 0–1.
11	from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score	Mengimpor fungsi metrik evaluasi: MSE, MAE, dan koefisien determinasi R^2.

3.2 Pemuatan dan Pra-Pemrosesan Dataset



Sel kedua memuat dataset, membersihkan data, membentuk deret waktu per jam, membuat sampel sliding window, melakukan normalisasi, dan mengubahnya menjadi tensor PyTorch.

Penjelasan :

No	Baris Kode	Penjelasan
1.	torch.manual_seed(42)	Menetapkan seed PyTorch agar inisialisasi bobot dan pengacakan batch dapat direproduksi.
2.	np.random.seed(42)	Menetapkan seed NumPy dengan tujuan yang sama.
3.	device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')	Memilih perangkat komputasi: GPU (cuda) jika tersedia, jika tidak memakai CPU.
4.	hasil = {}	Membuat dictionary kosong untuk menampung ringkasan hasil yang kelak disimpan ke JSON.
5.	# 1. Muat dataset	Komentar penanda tahap 1: memuat dataset.
6.	nama_file = 'Metro_Interstate_Traffic_Volume.csv'	Menyimpan nama file CSV dataset dalam variabel.
7.	if not os.path.exists(nama_file):	Memeriksa apakah file tersebut belum ada di direktori kerja.
8.	from google.colab import files	Jika belum ada (misalnya dijalankan di Google Colab), mengimpor modul files untuk fitur unggah.

No	Baris Kode	Penjelasan
9.	nama_file = list(files.upload().keys())[0]	Membuka dialog unggah file, lalu mengambil nama file pertama yang diunggah sebagai nama_file.
10.	df = pd.read_csv(nama_file)	Membaca file CSV menjadi DataFrame pandas bernama df.
11.	hasil['n_rows_raw'] = int(df.shape[0])	Menyimpan jumlah baris data mentah ke dictionary hasil.
12.	hasil['n_cols'] = int(df.shape[1])	Menyimpan jumlah kolom data mentah.
13.	hasil['columns'] = list(df.columns)	Menyimpan daftar nama kolom.
14.	print('Ukuran dataset (baris, kolom):', df.shape)	Menampilkan ukuran dataset (baris, kolom).
15.	print(df.head())	Menampilkan lima baris pertama untuk melihat struktur data.
16.	print(df.isnull().sum())	Menghitung dan menampilkan jumlah nilai kosong (missing) pada tiap kolom.
17.	# 2. Pembersihan data & pembentukan deret waktu per jam	Komentar penanda tahap 2: pembersihan data dan pembentukan deret waktu per jam.
18.	df['date_time'] = pd.to_datetime(df['date_time'])	Mengubah kolom date_time dari teks menjadi tipe datetime agar dapat diolah sebagai waktu.
19.	df = df.sort_values('date_time')	Mengurutkan data berdasarkan waktu secara menaik (kronologis).

No	Baris Kode	Penjelasan
20.	hasil['n_dupe_ts'] = int(df['date_time'].duplicated().sum())	Menghitung banyaknya timestamp yang duplikat dan menyimpannya.
21.	df = df.drop_duplicates(subset='date_time', keep='first')	Menghapus baris dengan timestamp duplikat dan mempertahankan kemunculan pertama, sehingga satu jam hanya memiliki satu nilai.
22.	series = df.set_index('date_time')['traffic_volume'].resample('h').mean()	Menjadikan date_time sebagai indeks, mengambil kolom target traffic_volume, lalu resample ke frekuensi per jam ("h"). Jam yang tidak tercatat menjadi NaN; mean dipakai jika ada lebih dari satu nilai dalam satu jam.
23.	hasil['n_hours_total'] = int(len(series))	Menyimpan jumlah total jam pada deret waktu.
24.	hasil['n_missing_before'] = int(series.isnull().sum())	Menyimpan jumlah jam yang kosong (NaN) sebelum interpolasi.

No	Baris Kode	Penjelasan
25.	series = series.interpolate(method='time', limit=3, limit_area='inside')	Mengisi nilai kosong dengan interpolasi linear berbasis waktu, maksimal 3 jam berturut-turut dan hanya di antara data valid (limit_area="inside"). Celah yang lebih panjang tidak dikarang nilainya.
26.	hasil['n_missing_after'] = int(series.isnull().sum())	Menyimpan jumlah jam kosong yang tersisa setelah interpolasi.
27.	hasil['date_start'] = str(series.index.min())	Menyimpan waktu awal deret.
28.	hasil['date_end'] = str(series.index.max())	Menyimpan waktu akhir deret.
29.	hasil['vol_min'] = float(series.min())	Menyimpan nilai minimum volume lalu lintas.
30.	hasil['vol_max'] = float(series.max())	Menyimpan nilai maksimum volume lalu lintas.
31.	hasil['vol_mean'] = float(series.mean())	Menyimpan rata-rata volume lalu lintas.
32.	hasil['vol_std'] = float(series.std())	Menyimpan simpangan baku volume lalu lintas.
33.	print(series.describe())	Menampilkan statistik deskriptif deret waktu (count, mean, std, min, kuartil, max).
34.	# 3. Sliding window: 24 jam terakhir -> prediksi 1 jam berikutnya	Komentar penanda tahap 3: pembuatan sliding window (24 jam terakhir → 1 jam berikutnya).

No	Baris Kode	Penjelasan
35.	WINDOW = 24	Menetapkan panjang window sebesar 24 jam, sesuai siklus harian.
36.	values = series.values.astype(float)	Mengambil nilai deret waktu sebagai array NumPy bertipe float.
37.	windows = np.lib.stride_tricks.sliding_window_view(values, WINDOW + 1)	Membuat jendela geser sepanjang WINDOW + 1 = 25 nilai (24 input + 1 target) tanpa menyalin memori. Hasilnya berbentuk (jumlah_jam − 24, 25).
38.	target_time = series.index[WINDOW:]	Menentukan timestamp target untuk tiap window, yaitu indeks waktu mulai dari jam ke-25.
39.	valid = ~np.isnan(windows).any(axis=1)	Membuat mask boolean: True untuk window yang tidak mengandung NaN sama sekali.
40.	windows = windows[valid]	Memfilter window sehingga hanya window valid yang dipakai.
41.	target_time = target_time[valid]	Memfilter timestamp target dengan mask yang sama agar tetap sejajar dengan window.

No	Baris Kode	Penjelasan
42.	split = int(len(windows) * 0.8)	Menentukan indeks pemisah 80% data latih dan 20% data uji. Pemisahan berurutan menjaga urutan waktu dan mencegah kebocoran data masa depan.
43.	# 4. Normalisasi Min-Max (fit hanya pada data latih)	Komentar penanda tahap 4: normalisasi Min-Max.
44.	scaler = MinMaxScaler()	Membuat objek MinMaxScaler.
45.	scaler.fit(windows[:split].reshape(-1, 1))	Menghitung nilai minimum dan maksimum hanya dari data latih (diubah menjadi satu kolom lewat reshape(−1, 1)) untuk mencegah data leakage.
46.	scaled = scaler.transform(windows.reshape(-1, 1)).reshape(windows.shape)	Menerapkan x' = (x − min) / (max − min) ke seluruh window, lalu mengembalikan ke bentuk semula.
47.	# 5. Konversi ke PyTorch Tensor (X: [sampel, 24, 1], y: [sampel, 1])	Komentar penanda tahap 5: konversi ke tensor PyTorch.
48.	X_train = torch.tensor(scaled[:split, :WINDOW], dtype=torch.float32).unsqueeze(-1)	Mengambil 24 kolom pertama sampel latih sebagai input, mengubahnya menjadi tensor float32, dan menambah dimensi fitur sehingga berbentuk [sampel, 24, 1].

No	Baris Kode	Penjelasan
49.	y_train = torch.tensor(scaled[:split, WINDOW], dtype=torch.float32).unsqueeze(-1)	Mengambil kolom ke-25 sampel latih sebagai target, berbentuk [sampel, 1].
50.	X_test = torch.tensor(scaled[split:, :WINDOW], dtype=torch.float32).unsqueeze(-1)	Membentuk tensor input data uji dengan cara yang sama.
51.	y_test = torch.tensor(scaled[split:, WINDOW], dtype=torch.float32).unsqueeze(-1)	Membentuk tensor target data uji.
52.	train_loader = DataLoader(TensorDataset(X_train, y_train), batch_size=64, shuffle=True)	Membuat DataLoader data latih dengan batch 64 dan shuffle=True (diacak tiap epoch; aman karena setiap sampel sudah berupa window yang berdiri sendiri).
53.	hasil['window'] = WINDOW	Menyimpan panjang window ke dictionary hasil.
54.	hasil['batch_size'] = 64	Menyimpan ukuran batch.
55.	hasil['n_windows'] = int(len(windows))	Menyimpan jumlah total window valid.
56.	hasil['n_train'] = int(len(X_train))	Menyimpan jumlah sampel latih.
57.	hasil['n_test'] = int(len(X_test))	Menyimpan jumlah sampel uji.
58.	hasil['test_start'] = str(target_time[split])	Menyimpan timestamp awal data uji.
59.	hasil['test_end'] = str(target_time[-1])	Menyimpan timestamp akhir data uji.
60.	print('Sampel latih:', X_train.shape, '| Sampel uji:', X_test.shape)	Menampilkan bentuk tensor sampel latih dan uji.

No	Baris Kode	Penjelasan
61.	print('Rentang uji:', hasil['test_start'], 's/d', hasil['test_end'])	Menampilkan rentang waktu data uji.

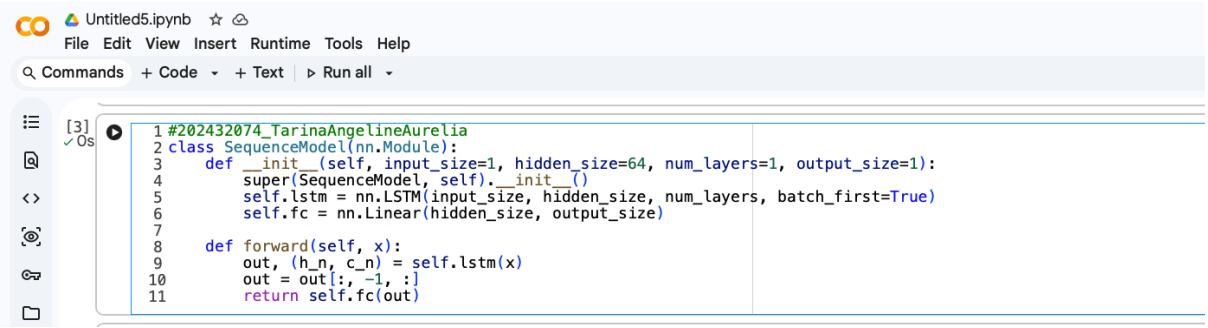
```
Ukuran dataset (baris, kolom): (48204, 9)
  holiday    temp  rain_1h  snow_1h  clouds_all weather_main  \
0     NaN  288.28      0.0      0.0          40       Clouds
1     NaN  289.36      0.0      0.0          75       Clouds
2     NaN  289.58      0.0      0.0          90       Clouds
3     NaN  290.13      0.0      0.0          90       Clouds
4     NaN  291.14      0.0      0.0          75       Clouds

  weather_description            date_time  traffic_volume
0    scattered clouds  2012-10-02 09:00:00            5545
1       broken clouds  2012-10-02 10:00:00            4516
2     overcast clouds  2012-10-02 11:00:00            4767
3     overcast clouds  2012-10-02 12:00:00            5026
4       broken clouds  2012-10-02 13:00:00            4918
holiday                48143
temp                       0
rain_1h                    0
snow_1h                    0
clouds_all                 0
weather_main               0
weather_description        0
date_time                  0
traffic_volume             0
dtype: int64
count    43754.000000
mean      3304.057520
std       1974.229144
min          0.000000
25%       1304.000000
50%       3455.000000
75%       4961.000000
max       7280.000000
Name: traffic_volume, dtype: float64
Sampel latih: torch.Size([33428, 24, 1]) | Sampel uji: torch.Size([8358, 24, 1])
Rentang uji: 2017-10-16 15:00:00 s/d 2018-09-30 23:00:00
```

Gambar 2. Output sel pemuatan dan pra-pemrosesan data

3.3 Perancangan Arsitektur Model LSTM

Sel berikut mendefinisikan arsitektur jaringan. Aliran data pada model: input [batch, 24, 1] → LSTM (64 unit hidden) → ambil hidden state langkah terakhir [batch, 64] → Linear(64 → 1) → prediksi [batch, 1].



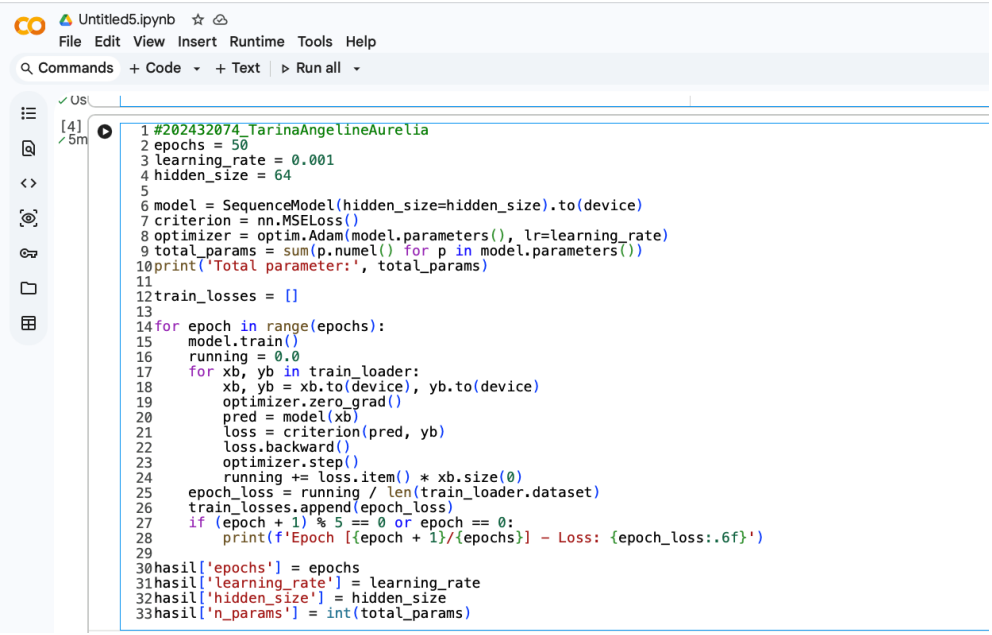
Penjelasan :

No	Baris Kode	Penjelasan
1.	class SequenceModel(nn.Module):	Mendefinisikan kelas SequenceModel yang mewarisi nn.Module, kelas dasar untuk semua jaringan saraf PyTorch.
2.	def __init__(self, input_size=1, hidden_size=64, num_layers=1, output_size=1):	Konstruktor dengan parameter bawaan: 1 fitur input per langkah waktu, 64 unit hidden, 1 layer LSTM, dan 1 output.
3.	super(SequenceModel, self).__init__()	Memanggil konstruktor kelas induk agar nn.Module terinisialisasi dan dapat mendaftarkan parameter.
4.	self.lstm = nn.LSTM(input_size, hidden_size, num_layers, batch_first=True)	Membuat layer LSTM. batch_first=True berarti bentuk input adalah (batch, waktu, fitur). Di dalamnya terdapat gerbang forget, input, candidate, dan output seperti pada Bab 2.
5.	self.fc = nn.Linear(hidden_size, output_size)	Membuat layer Linear (dense) yang memetakan 64 nilai hidden state menjadi 1 nilai prediksi.
6.	def forward(self, x):	Mendefinisikan alur maju (forward pass) model; x adalah batch input berbentuk [batch, 24, 1].
7.	out, (h_n, c_n) = self.lstm(x)	Menjalankan LSTM pada seluruh 24 langkah waktu. out berisi hidden state setiap langkah [batch, 24, 64]; h_n dan c_n adalah hidden state dan cell state terakhir.

No	Baris Kode	Penjelasan
8.	out = out[:, -1, :]	Mengambil hidden state langkah terakhir (jam ke-24) sehingga berbentuk [batch, 64]; ini ringkasan seluruh sekuens.
9.	return self.fc(out)	Memasukkan ringkasan tersebut ke layer Linear dan mengembalikan prediksi (masih pada skala ternormalisasi).

3.4 Konfigurasi dan Proses Pelatihan

Sel berikut menetapkan hyperparameter, membuat model, loss function, dan optimizer, lalu menjalankan siklus pelatihan selama 50 epoch.



Penjelasan setiap baris:

No	Baris Kode	Penjelasan
1.	epochs = 50	Jumlah epoch: seluruh data latih dilewatkan ke model sebanyak 50 kali.
2.	learning_rate = 0.001	Learning rate 0,001, yaitu ukuran langkah pembaruan bobot oleh optimizer.
3.	hidden_size = 64	Jumlah unit hidden pada LSTM sebesar 64.

No	Baris Kode	Penjelasan
4.	model = SequenceModel(hidden_size=hidden_size).to(device)	Membuat objek model dengan hidden_size tersebut dan memindahkannya ke perangkat (GPU/CPU).
5.	criterion = nn.MSELoss()	Menentukan loss function Mean Squared Error, yaitu rata-rata kuadrat selisih prediksi dan nilai sebenarnya.
6.	optimizer = optim.Adam(model.parameters(), lr=learning_rate)	Membuat optimizer Adam yang akan memperbarui seluruh parameter model dengan learning rate yang ditentukan.
7.	total_params = sum(p.numel() for p in model.parameters())	Menghitung total parameter model dengan menjumlahkan jumlah elemen setiap parameter (seharusnya 17.217).
8.	print('Total parameter:', total_params)	Menampilkan total parameter.
9.	train_losses = []	Membuat list kosong untuk menyimpan rata-rata loss setiap epoch.
10.	for epoch in range(epochs):	Perulangan utama: satu iterasi untuk setiap epoch.
11.	model.train()	Mengaktifkan mode training pada model.
12.	running = 0.0	Menginisialisasi akumulator loss epoch ini dengan 0.
13.	for xb, yb in train_loader:	Perulangan mini-batch: mengambil pasangan input (xb) dan target (yb) dari train_loader.

No	Baris Kode	Penjelasan
14.	xb, yb = xb.to(device), yb.to(device)	Memindahkan batch ke perangkat yang sama dengan model.
15.	optimizer.zero_grad()	Mengosongkan gradien hasil iterasi sebelumnya (PyTorch mengakumulasi gradien secara bawaan).
16.	pred = model(xb)	Forward pass: model menghasilkan prediksi untuk batch ini.
17.	loss = criterion(pred, yb)	Menghitung nilai loss (MSE) antara prediksi dan target.
18.	loss.backward()	Backward pass: menghitung gradien loss terhadap seluruh parameter (backpropagation through time).
19.	optimizer.step()	Optimizer memperbarui bobot berdasarkan gradien tersebut.
20.	running += loss.item() * xb.size(0)	Menambahkan loss batch dikalikan ukuran batch ke akumulator, agar batch terakhir yang lebih kecil terbobot benar.
21.	epoch_loss = running / len(train_loader.dataset)	Menghitung rata-rata loss epoch dengan membagi akumulator dengan jumlah seluruh sampel latih.
22.	train_losses.append(epoch_loss)	Menyimpan loss epoch ke list train_losses untuk dibuat grafik.
23.	if (epoch + 1) % 5 == 0 or epoch == 0:	Kondisi pencetakan log: pada epoch pertama dan setiap kelipatan 5.

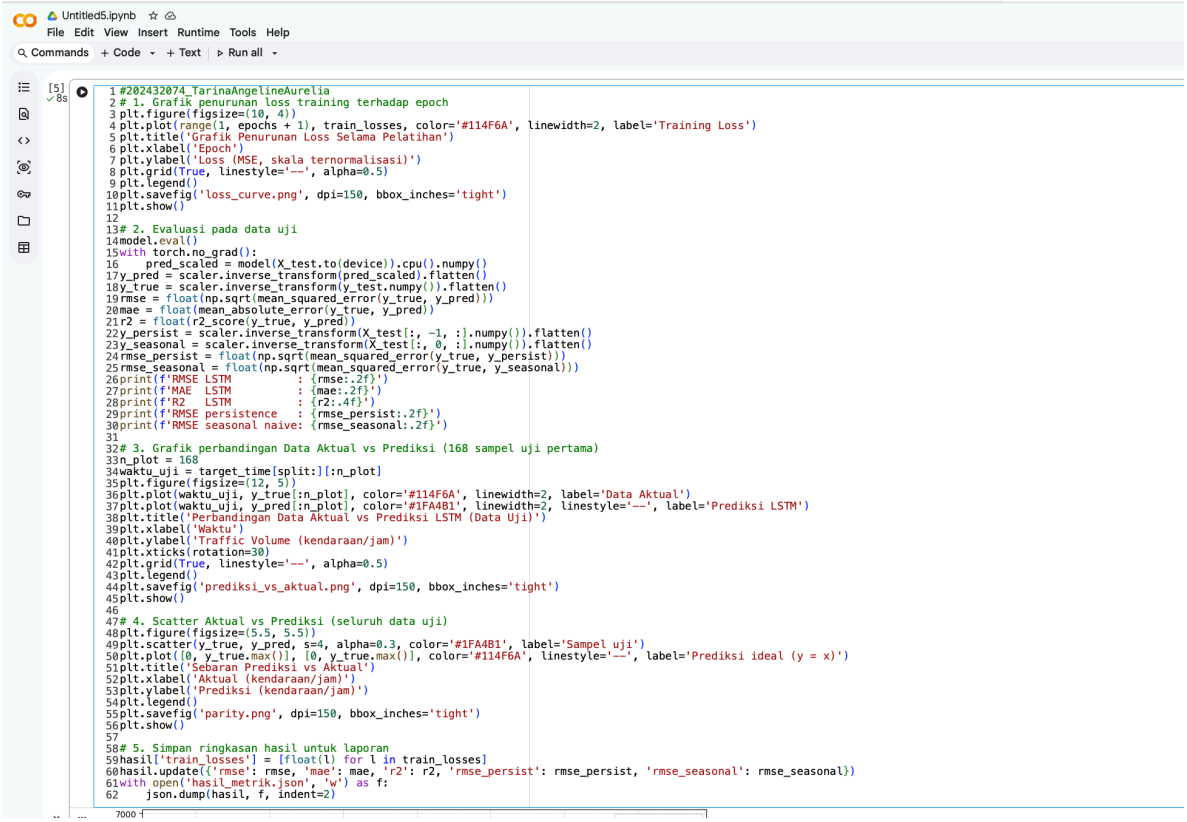
No	Baris Kode	Penjelasan
24.	print(f'Epoch [{epoch + 1}/{epochs}] - Loss: {epoch_loss:.6f}')	Mencetak nomor epoch dan nilai loss dengan 6 angka desimal.
25.	hasil['epochs'] = epochs	Menyimpan jumlah epoch ke dictionary hasil.
26.	hasil['learning_rate'] = learning_rate	Menyimpan learning rate.
27.	hasil['hidden_size'] = hidden_size	Menyimpan hidden size.
28.	hasil['n_params'] = int(total_params)	Menyimpan jumlah parameter model.

Output :

```
Total parameter: 17217
Epoch [1/50] - Loss: 0.021812
Epoch [5/50] - Loss: 0.002691
Epoch [10/50] - Loss: 0.002020
Epoch [15/50] - Loss: 0.001840
Epoch [20/50] - Loss: 0.001736
Epoch [25/50] - Loss: 0.001664
Epoch [30/50] - Loss: 0.001604
Epoch [35/50] - Loss: 0.001572
Epoch [40/50] - Loss: 0.001529
Epoch [45/50] - Loss: 0.001489
Epoch [50/50] - Loss: 0.001491
```

3.5 Visualisasi Performa dan Evaluasi Model

Sel terakhir menampilkan grafik riwayat loss, mengevaluasi model pada data uji, membandingkannya dengan dua baseline, serta membuat grafik prediksi.



```
#202432074_TarinaAngelineAurelia
# 1. Grafik penurunan loss training terhadap epoch
plt.figure(figsize=(10, 4))
plt.plot(range(1, epochs + 1), train_losses, color='#114F6A', linewidth=2, label='Training Loss')
plt.title('Grafik Penurunan Loss Selama Pelatihan')
plt.xlabel('Epoch')
plt.ylabel('Loss (MSE, skala ternormalisasi)')
plt.grid(True, linestyle='--', alpha=0.5)
plt.legend()
plt.savefig('loss_curve.png', dpi=150, bbox_inches='tight')
plt.show()

# 2. Evaluasi pada data uji
model.eval()
with torch.no_grad():
    pred_scaled = model(X_test.to(device)).cpu().numpy()
y_pred = scaler.inverse_transform(pred_scaled).flatten()
y_true = scaler.inverse_transform(y_test.numpy()).flatten()
rmse = float(np.sqrt(mean_squared_error(y_true, y_pred)))
mae = float(mean_absolute_error(y_true, y_pred))
r2 = float(r2_score(y_true, y_pred))
y_persist = scaler.inverse_transform(X_test[:, -1, :].numpy()).flatten()
y_seasonal = scaler.inverse_transform(X_test[:, 0, :].numpy()).flatten()
rmse_persist = float(np.sqrt(mean_squared_error(y_true, y_persist)))
rmse_seasonal = float(np.sqrt(mean_squared_error(y_true, y_seasonal)))
print(f'RMSE LSTM           : {rmse:.2f}')
print(f'MAE  LSTM           : {mae:.2f}')
print(f'R2   LSTM           : {r2:.4f}')
print(f'RMSE persistence    : {rmse_persist:.2f}')
print(f'RMSE seasonal naive: {rmse_seasonal:.2f}')

# 3. Grafik perbandingan Data Aktual vs Prediksi (168 sampel uji pertama)
n_plot = 168
waktu_uji = target_time[split:][:n_plot]
plt.figure(figsize=(12, 5))
plt.plot(waktu_uji, y_true[:n_plot], color='#114F6A', linewidth=2, label='Data Aktual')
plt.plot(waktu_uji, y_pred[:n_plot], color='#1FA4B1', linewidth=2, linestyle='--', label='Prediksi LSTM')
plt.title('Perbandingan Data Aktual vs Prediksi LSTM (Data Uji)')
plt.xlabel('Waktu')
plt.ylabel('Traffic Volume (kendaraan/jam)')
plt.xticks(rotation=30)
plt.grid(True, linestyle='--', alpha=0.5)
plt.legend()
plt.savefig('prediksi_vs_aktual.png', dpi=150, bbox_inches='tight')
plt.show()

# 4. Scatter Aktual vs Prediksi (seluruh data uji)
plt.figure(figsize=(5.5, 5.5))
plt.scatter(y_true, y_pred, s=4, alpha=0.3, color='#1FA4B1', label='Sampel uji')
plt.plot([0, y_true.max()], [0, y_true.max()], color='#114F6A', linestyle='--', label='Prediksi ideal (y = x)')
plt.title('Sebaran Prediksi vs Aktual')
plt.xlabel('Aktual (kendaraan/jam)')
plt.ylabel('Prediksi (kendaraan/jam)')
plt.legend()
plt.savefig('parity.png', dpi=150, bbox_inches='tight')
plt.show()

# 5. Simpan ringkasan hasil untuk laporan
hasil['train_losses'] = [float(l) for l in train_losses]
hasil.update({'rmse': rmse, 'mae': mae, 'r2': r2, 'rmse_persist': rmse_persist, 'rmse_seasonal': rmse_seasonal})
with open('hasil_metrik.json', 'w') as f:
    json.dump(hasil, f, indent=2)
```

Penjelasan setiap baris:

No	Baris Kode	Penjelasan
1.	# 1. Grafik penurunan loss training terhadap epoch	Komentar penanda bagian 1: grafik penurunan loss.
2.	plt.figure(figsize=(10, 4))	Membuat kanvas grafik berukuran 10 × 4 inci.
3.	plt.plot(range(1, epochs + 1), train_losses, color='#114F6A', linewidth=2, label='Training Loss')	Menggambar kurva loss (sumbu x: epoch 1–50, sumbu y: train_losses) dengan warna dan ketebalan garis tertentu serta label legenda.
4.	plt.title('Grafik Penurunan Loss Selama Pelatihan')	Memberi judul grafik.
5.	plt.xlabel('Epoch')	Memberi label sumbu x: Epoch.

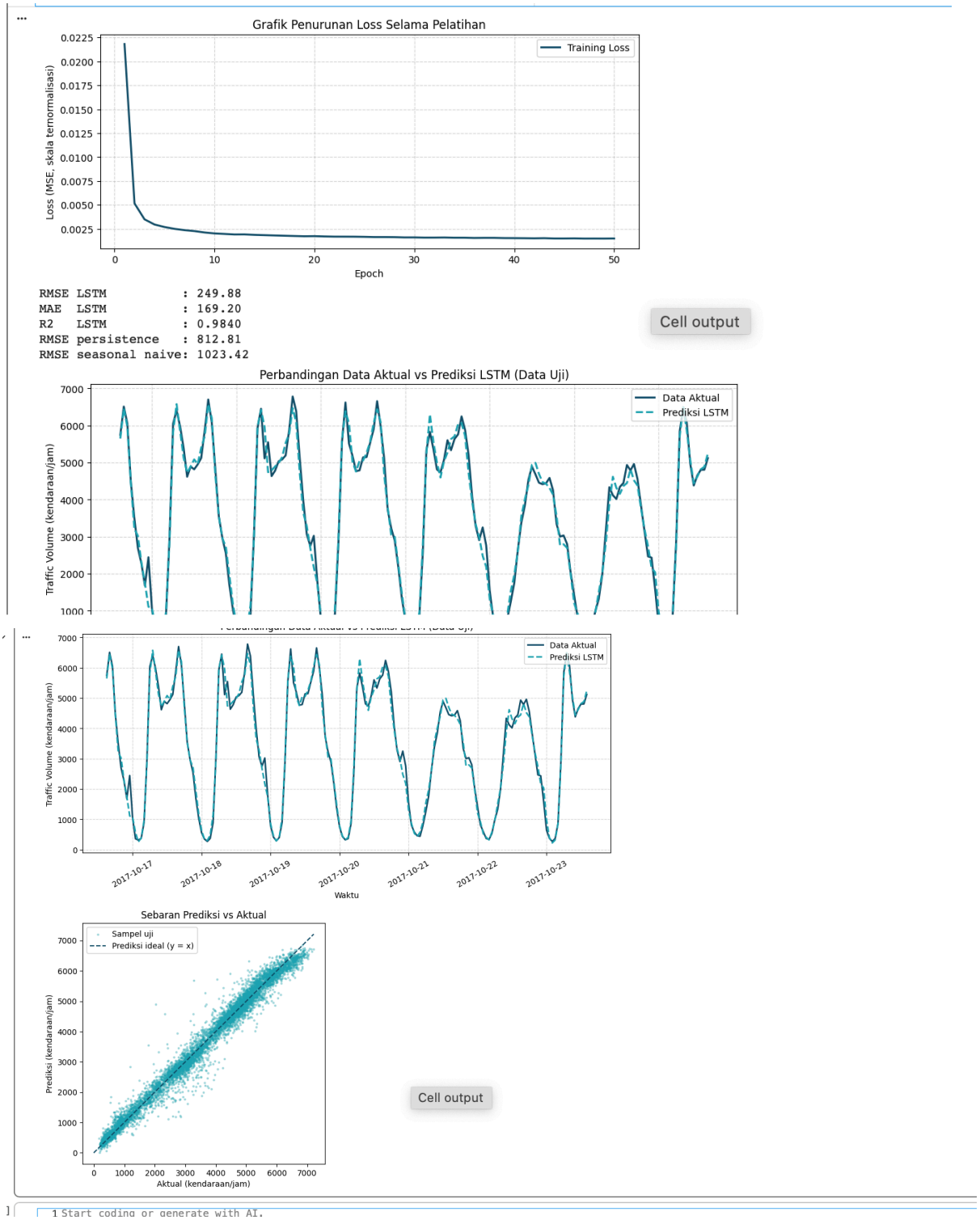
No	Baris Kode	Penjelasan
6.	plt.ylabel('Loss (MSE, skala ternormalisasi)')	Memberi label sumbu y: Loss (MSE) pada skala ternormalisasi.
7.	plt.grid(True, linestyle='--', alpha=0.5)	Menampilkan garis grid putus-putus agar nilai lebih mudah dibaca.
8.	plt.legend()	Menampilkan legenda.
9.	plt.savefig('loss_curve.png', dpi=150, bbox_inches='tight')	Menyimpan grafik sebagai berkas gambar loss_curve.png (resolusi 150 dpi, margin dirapikan).
10.	plt.show()	Menampilkan grafik di notebook.
11.	# 2. Evaluasi pada data uji	Komentar penanda bagian 2: evaluasi pada data uji.
12.	model.eval()	Mengubah model ke mode evaluasi (menonaktifkan perilaku khusus training).
13.	with torch.no_grad():	Menonaktifkan perhitungan gradien karena hanya melakukan inferensi; menghemat memori dan waktu.
14.	pred_scaled = model(X_test.to(device)).cpu().numpy()	Memprediksi seluruh data uji, memindahkan hasil ke CPU, dan mengubahnya menjadi array NumPy (masih skala ternormalisasi).
15.	y_pred = scaler.inverse_transform(pred_scaled).flatten()	Mengembalikan prediksi ke satuan asli (kendaraan/jam) dengan inverse_transform, lalu meratakannya menjadi 1 dimensi.
16.	y_true = scaler.inverse_transform(y_test.numpy()).flatten()	Mengembalikan target sebenarnya data uji ke satuan asli.
17.	rmse = float(np.sqrt(mean_squared_error(y_true, y_pred)))	Menghitung RMSE = akar dari MSE antara nilai aktual dan prediksi.

No	Baris Kode	Penjelasan
18.	mae = float(mean_absolute_error(y_true, y_pred))	Menghitung MAE, yaitu rata-rata selisih absolut.
19.	r2 = float(r2_score(y_true, y_pred))	Menghitung R^2, yaitu proporsi variasi data yang dijelaskan model.
20.	y_persist = scaler.inverse_transform(X_test[:, -1, :].numpy()).flatten()	Membuat baseline persistence: prediksi jam berikutnya = nilai jam terakhir (langkah ke-24 pada window), dikembalikan ke satuan asli.
21.	y_seasonal = scaler.inverse_transform(X_test[:, 0, :].numpy()).flatten()	Membuat baseline seasonal naive: prediksi = nilai pada jam yang sama kemarin (langkah pertama window, 24 jam sebelumnya).
22.	rmse_persist = float(np.sqrt(mean_squared_error(y_true, y_persist)))	Menghitung RMSE baseline persistence.
23.	rmse_seasonal = float(np.sqrt(mean_squared_error(y_true, y_seasonal)))	Menghitung RMSE baseline seasonal naive.
24.	print(f'RMSE LSTM : {rmse:.2f}')	Mencetak RMSE model LSTM.
25.	print(f'MAE LSTM : {mae:.2f}')	Mencetak MAE model LSTM.
26.	print(f'R2 LSTM : {r2:.4f}')	Mencetak R^2 model LSTM.
27.	print(f'RMSE persistence : {rmse_persist:.2f}')	Mencetak RMSE baseline persistence.
28.	print(f'RMSE seasonal naive: {rmse_seasonal:.2f}')	Mencetak RMSE baseline seasonal naive.
29.	# 3. Grafik perbandingan Data Aktual vs Prediksi (168 sampel uji pertama)	Komentar penanda bagian 3: grafik aktual vs prediksi.
30.	n_plot = 168	Menentukan jumlah sampel yang digambar, yaitu 168 jam (7 hari) pertama data uji.

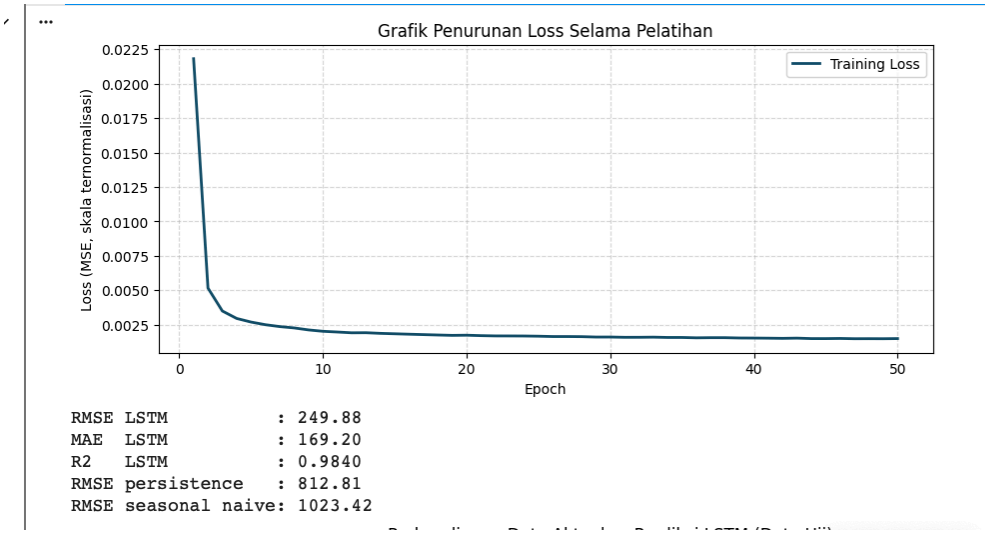
No	Baris Kode	Penjelasan
31.	waktu_uji = target_time[split:][:n_plot]	Mengambil timestamp untuk 168 sampel uji pertama sebagai sumbu x.
32.	plt.figure(figsize=(12, 5))	Membuat kanvas grafik berukuran 12 × 5 inci.
33.	plt.plot(waktu_uji, y_true[:n_plot], color='#114F6A', linewidth=2, label='Data Aktual')	Menggambar kurva data aktual (garis solid).
34.	plt.plot(waktu_uji, y_pred[:n_plot], color='#1FA4B1', linewidth=2, linestyle='--', label='Prediksi LSTM')	Menggambar kurva prediksi LSTM (garis putus-putus) pada sumbu yang sama.
35.	plt.title('Perbandingan Data Aktual vs Prediksi LSTM (Data Uji)')	Memberi judul grafik.
36.	plt.xlabel('Waktu')	Memberi label sumbu x: Waktu.
37.	plt.ylabel('Traffic Volume (kendaraan/jam)')	Memberi label sumbu y: Traffic Volume (kendaraan/jam).
38.	plt.xticks(rotation=30)	Memutar label sumbu x 30° agar tidak saling menimpa.
39.	plt.grid(True, linestyle='--', alpha=0.5)	Menampilkan grid.
40.	plt.legend()	Menampilkan legenda.
41.	plt.savefig('prediksi_vs_aktual.png', dpi=150, bbox_inches='tight')	Menyimpan grafik sebagai prediksi_vs_aktual.png.
42.	plt.show()	Menampilkan grafik.
43.	# 4. Scatter Aktual vs Prediksi (seluruh data uji)	Komentar penanda bagian 4: scatter aktual vs prediksi.
44.	plt.figure(figsize=(5.5, 5.5))	Membuat kanvas persegi berukuran 5,5 × 5,5 inci.
45.	plt.scatter(y_true, y_pred, s=4, alpha=0.3, color='#1FA4B1', label='Sampel uji')	Menggambar titik-titik (aktual pada sumbu x, prediksi pada sumbu y) untuk seluruh data uji dengan ukuran kecil dan transparansi 0,3.
46.	plt.plot([0, y_true.max()], [0, y_true.max()], color='#114F6A', linestyle='--', label='Prediksi ideal (y = x)')	Menggambar garis diagonal y = x sebagai acuan prediksi sempurna.

No	Baris Kode	Penjelasan
47.	plt.title('Sebaran Prediksi vs Aktual')	Memberi judul grafik.
48.	plt.xlabel('Aktual (kendaraan/jam)')	Memberi label sumbu x: nilai aktual.
49.	plt.ylabel('Prediksi (kendaraan/jam)')	Memberi label sumbu y: nilai prediksi.
50.	plt.legend()	Menampilkan legenda.
51.	plt.savefig('parity.png', dpi=150, bbox_inches='tight')	Menyimpan grafik sebagai parity.png.
52.	plt.show()	Menampilkan grafik.
53.	# 5. Simpan ringkasan hasil untuk laporan	Komentar penanda bagian 5: menyimpan ringkasan hasil.
54.	hasil['train_losses'] = [float(l) for l in train_losses]	Menyimpan seluruh nilai loss per epoch ke dictionary hasil (dikonversi ke float agar bisa ditulis ke JSON).
55.	hasil.update({'rmse': rmse, 'mae': mae, 'r2': r2, 'rmse_persist': rmse_persist, 'rmse_seasonal': rmse_seasonal})	Menambahkan metrik RMSE, MAE, R^2, dan RMSE kedua baseline ke dictionary hasil.
56.	with open('hasil_metrik.json', 'w') as f:	Membuka (membuat) berkas hasil_metrik.json dalam mode tulis.
57.	json.dump(hasil, f, indent=2)	Menulis dictionary hasil ke berkas JSON dengan indentasi 2 spasi.

3.6 Hasil dan Analisis Grafik Performa



3.6.1 Grafik Riwayat Loss Pelatihan



Gambar 3. Grafik penurunan loss pelatihan terhadap epoch

Gambar 3 memperlihatkan nilai loss (MSE pada skala ternormalisasi) pada setiap epoch. Loss pada epoch pertama bernilai [loss epoch 1] dan turun menjadi [loss epoch 50] pada epoch ke-50. Penurunan yang tajam pada epoch awal lalu melandai menunjukkan bahwa model dengan cepat mempelajari pola utama (siklus harian) dan selanjutnya hanya melakukan penyesuaian halus. [Tulis pengamatan: kurva sudah mendatar sehingga konvergen / masih menurun sehingga epoch dapat ditambah]. Karena yang dicatat hanya loss data latih, grafik ini menunjukkan proses belajar model, sedangkan kemampuan generalisasi dinilai dari metrik pada data uji.

3.6.2 Hasil Evaluasi pada Data Uji

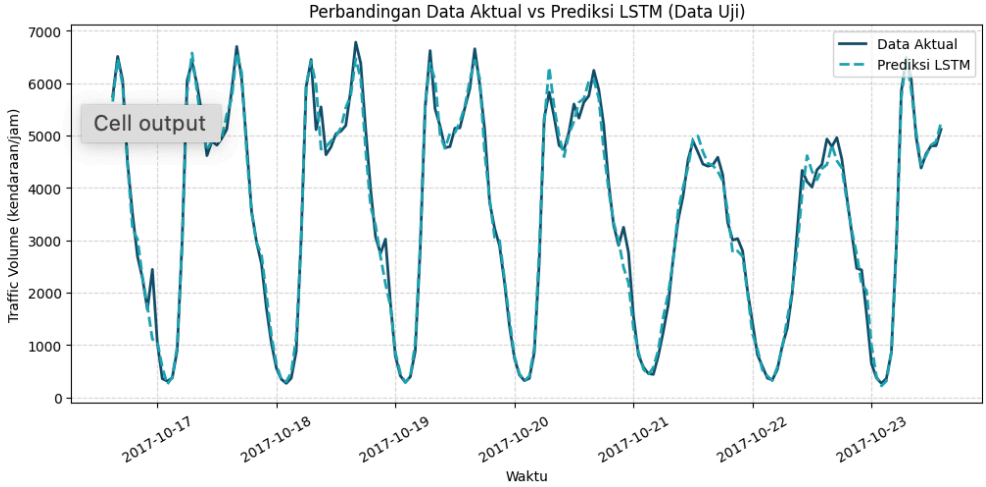
Tabel 8. Metrik evaluasi pada data uji

Model	RMSE (kend./jam)	MAE (kend./jam)	R^2
LSTM	[rmse]	[mae]	[r2]
Baseline persistence (nilai 1 jam lalu)	[rmse_persist]	–	–
Baseline seasonal naive (nilai 24 jam lalu)	[rmse_seasonal]	–	–

RMSE dan MAE dinyatakan dalam satuan kendaraan per jam sehingga mudah dibandingkan langsung dengan rata-rata volume lalu lintas ([vol_mean] kendaraan/jam); RMSE model setara dengan [RMSE ÷ vol_mean × 100]% dari rata-rata tersebut. Nilai R^2 sebesar [r2] berarti model menjelaskan sekitar [r2 × 100]% variasi volume lalu lintas pada data

uji. Dibandingkan dua baseline, RMSE LSTM [lebih kecil / lebih besar] daripada persistence ([rmse_persist]) dan seasonal naive ([rmse_seasonal]). [Tulis interpretasi: jika LSTM lebih kecil, model memberi nilai tambah dibanding aturan sederhana; jika tidak, jelaskan bahwa prediksi 1 jam ke depan pada data per jam sangat kuat ditebak dari nilai jam sebelumnya].

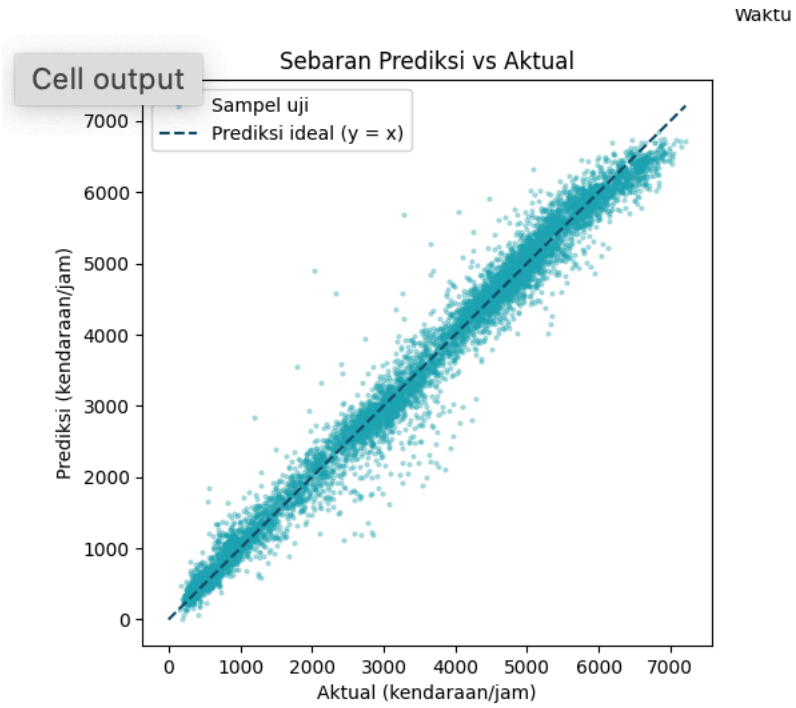
3.6.3 Grafik Data Aktual vs Prediksi



Gambar 4. Perbandingan data aktual dan prediksi LSTM pada 168 jam pertama data uji

Gambar 4 membandingkan volume aktual dengan prediksi LSTM selama 168 jam (7 hari) pertama data uji. Pola lalu lintas jalan arteri umumnya memperlihatkan dua puncak pada hari kerja (jam sibuk pagi dan sore) dan volume yang lebih rendah pada akhir pekan. Pada grafik ini, kurva prediksi [mengikuti / kurang mengikuti] pola tersebut: [tulis pengamatan pada puncak pagi, puncak sore, lembah dini hari, dan akhir pekan]. Selisih terbesar antara kedua kurva terlihat pada [tulis lokasi/waktu selisih terbesar, mis. puncak tertinggi atau perubahan mendadak].

3.6.4 Grafik Sebaran Prediksi vs Aktual



Gambar 5. Sebaran prediksi terhadap nilai aktual pada seluruh data uji

Gambar 5 menggambarkan setiap sampel uji sebagai titik; semakin dekat titik ke garis diagonal y = x, semakin akurat prediksi. Sebaran titik [rapat di sekitar garis / menyebar], dengan penyimpangan yang paling terlihat pada [tulis: volume rendah / sedang / tinggi]. Hal ini [konsisten / tidak konsisten] dengan nilai R^2 pada Tabel 8.

BAB 4
KESIMPULAN TEORI & PRAKTIKUM

Berdasarkan keseluruhan pembahasan, kesimpulan yang dapat ditarik adalah sebagai berikut.

1. **Keterkaitan teori dan implementasi.** Perhitungan manual pada Bab 2 menunjukkan bahwa satu langkah LSTM terdiri dari forget gate, input gate, candidate, pembaruan cell state, output gate, dan hidden state. Pada contoh dengan $x_t = 0{,}6$ diperoleh $h_t = 0.3968$. Seluruh perhitungan tersebut dilakukan oleh nn.LSTM pada Bab 3, tetapi dengan 64 unit hidden, bobot hasil pelatihan, dan diulang selama 24 langkah waktu untuk setiap sampel.
2. **Alur data.** Konsep sekuens pada teori diwujudkan melalui sliding window: 24 jam terakhir (24 langkah waktu, 1 fitur) menjadi input dan volume jam ke-25 menjadi target. Hidden state langkah ke-24 diambil sebagai ringkasan sekuens dan dipetakan oleh lapisan Linear menjadi prediksi, kemudian dikembalikan ke satuan kendaraan/jam dengan kebalikan normalisasi Min-Max.
3. **Hasil pelatihan.** Loss pelatihan turun dari [loss epoch 1] menjadi [loss epoch 50] selama 50 epoch, yang menunjukkan bahwa mekanisme gerbang dan cell state pada LSTM dapat dilatih secara stabil pada data deret waktu lalu lintas.
4. **Hasil evaluasi.** Pada data uji, model mencapai RMSE [rmse], MAE [mae], dan R^2 [r2]. Dibandingkan baseline persistence ([rmse_persist]) dan seasonal naive ([rmse_seasonal]), kinerja LSTM [lebih baik / belum lebih baik]. [Tulis satu kalimat kesimpulan akhir berdasarkan Tabel 8 dan Gambar 4–5.]
5. **Keterbatasan dan saran.** Model bersifat univariat sehingga belum memanfaatkan fitur cuaca dan hari libur; pelatihan hanya memantau loss data latih tanpa data validasi; dan hasil berasal dari satu kali pelatihan dengan satu nilai seed. Pengembangan berikutnya dapat menambahkan fitur eksogen (cuaca, hari libur, jam dan hari dalam minggu), menambah data validasi dengan early stopping, menguji beberapa nilai window dan hidden size, serta membandingkan dengan RNN atau GRU.

DAFTAR PUSTAKA

Bengio, Y., Simard, P., & Frasconi, P. (1994). Learning long-term dependencies with gradient descent is difficult. IEEE Transactions on Neural Networks, 5(2), 157–166.

Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. Neural Computation, 9(8), 1735–1780.

Hogue, J. (2019). Metro Interstate Traffic Volume [Dataset]. UCI Machine Learning Repository.

Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. International Conference on Learning Representations (ICLR).

Kaggle. Metro Interstate Traffic Volume. https://www.kaggle.com/code/ramyahr/metro-interstate-traffic-volume/data

Paszke, A., dkk. (2019). PyTorch: An imperative style, high-performance deep learning library. Advances in Neural Information Processing Systems 32.

CATATAN PENGECEKAN (HAPUS HALAMAN INI SEBELUM DIJADIKAN PDF)

Bagian yang ditandai kuning [seperti ini] harus diisi dari hasil menjalankan notebook (Run All). Setelah semua terisi, hilangkan sorotan kuning, hapus halaman ini, lalu simpan sebagai PDF dengan nama UTS_DEEPL_B_202432074_TARINAANGELINEAURELIA.pdf.

- Tautan repositori GitHub (sampul, subbab 1.3.1, dan awal Bab 3).
- Tangkapan layar halaman dataset di Kaggle (Gambar 1).
- Tangkapan layar output sel pra-pemrosesan (Gambar 2) dan angka pada Tabel 6 (ada di hasil_metrik.json).
- Nilai loss per epoch pada Tabel 7 (dari output sel pelatihan).
- Gambar loss_curve.png, prediksi_vs_aktual.png, dan parity.png (Gambar 3–5), dan angka metrik pada Tabel 8 (dari output sel visualisasi).
- Kalimat analisis pada subbab 3.6 dan poin hasil pada Bab 4: pilih opsi yang sesuai dengan grafik dan angka Anda.